

Задание 1. Рефакторинг системы управления онлайн-курсами

1. Постановка задачи

Условие

Описание системы

Рассматривается система управления онлайн-курсами. Система поддерживает создание курсов, регистрацию студентов, прохождение обучения, расчет итоговой оценки, отправку уведомлений и оплату курсов через внешний сервис.

Текущая реализация построена вокруг класса `CourseManager`.

Класс `CourseManager` выполняет следующие функции:

- создает объекты курсов разных типов (например, `OnlineCourse`, `OfflineCourse`);
- хранит данные о курсах и пользователях;
- рассчитывает итоговую оценку студентов;
- отправляет уведомления (например, через вызовы `sendEmail(...)`);
- выполняет оплату через прямой вызов внешнего API.

Проблемы текущей реализации

В системе наблюдаются следующие недостатки.

1. В классе `CourseManager` сосредоточены разнородные обязанности. В одном месте находятся создание курсов, бизнес-логика, уведомления и работа с внешними сервисами.
2. Создание курсов реализовано через условные конструкции и прямые вызовы конструкторов:

```
if (type == "online") new OnlineCourse(...)
else if (type == "offline") new OfflineCourse(...)
```

При добавлении нового типа курса необходимо изменять существующий код.

1. Алгоритм расчета оценки жестко зафиксирован в одном методе. Он используется для всех курсов, и изменить его без изменения класса невозможно.
2. Отправка уведомлений встроена в бизнес-логику. Вызовы вроде `sendEmail(...)` находятся прямо внутри методов. Добавление другого канала уведомлений требует изменения существующего кода.
3. Работа с платежной системой реализована напрямую. В коде используются конкретные классы внешнего API, а их методы вызываются из бизнес-логики. При смене платежного сервиса потребуется изменить код системы.
4. В коде присутствует дублирование логики. Похожие действия выполняются в разных методах `CourseManager`.

Требуемый результат

Необходимо предложить новую структуру системы, в которой:

- обязанности распределены между отдельными классами и интерфейсами;
- создание объектов не зависит от конкретных реализаций;
- алгоритмы могут изменяться независимо от основной логики;
- механизм уведомлений не встроен напрямую в бизнес-логику;
- взаимодействие с платежной системой изолировано;
- дублирование логики устранено или сведено к минимуму.

Требования к решению

В работе необходимо:

1. Выполнить краткий анализ исходной системы и указать, какие из перечисленных проблем в ней присутствуют.
2. Предложить новую структуру классов и их взаимодействий.
3. Использовать не менее четырех паттернов проектирования из изученного списка (например: Factory Method, Abstract Factory, Adapter, Strategy, Observer, Proxy, Command).
4. Для каждого используемого паттерна указать:
 5. где он используется в системе;
 6. какую задачу он решает.
7. Показать соответствие между проблемами и решениями. Для каждой проблемы нужно указать, за счет какого изменения в архитектуре она устраняется.
8. Соблюдать принципы DRY, KISS и разделения ответственности.

Формат сдачи

Работа должна включать:

1. Краткий анализ исходной системы.
2. Диаграмму классов (UML или аналог).
3. Описание основных классов и их ответственности.
4. Таблицу следующего вида:

Проблема	Решение	Используемый паттерн	Обоснование
----------	---------	----------------------	-------------

5. Краткий вывод.

Код или псевдокод можно добавить, но это не обязательно.

Условие зачета паттернов

Паттерн учитывается при оценке только в том случае, если он действительно отражен в архитектуре и используется для решения конкретной проблемы системы.

Простое упоминание паттерна без его роли в решении не учитывается.

1. Краткий анализ исходной системы

Исходная реализация построена вокруг перегруженного класса `CourseManager`. Такой класс фактически является God Object: он создает курсы, хранит данные, рассчитывает оценки, отправляет уведомления и напрямую вызывает внешний платежный API. Это нарушает разделение ответственности, принцип открытости/закрытости и инверсию зависимостей: бизнес-логика зависит от конкретных классов, конструкторов и внешних API.

- `CourseManager` имеет слишком много обязанностей: управление курсами, пользователями, оценками, уведомлениями и платежами находится в одном месте.
- Создание курсов реализовано через `if/else` и прямые вызовы конструкторов `OnlineCourse`, `OfflineCourse`.
- Добавление нового типа курса требует изменения существующего `CourseManager`, то есть система плохо расширяется.
- Алгоритм расчета итоговой оценки жестко зафиксирован в одном методе и применяется ко всем курсам одинаково.
- Уведомления встроены в бизнес-логику через прямые вызовы `sendEmail(...)`. Добавление SMS или Telegram потребует менять бизнес-код.
- Платежный внешний API вызывается напрямую из бизнес-логики. Замена провайдера оплаты затронет основной код системы.
- Похожие действия повторяются в разных методах: проверки, сохранение данных, отправка уведомлений, обработка результата оплаты.

2. Предлагаемая структура системы

Вместо одного `CourseManager` система делится на небольшие сервисы, интерфейсы, фабрики, стратегии и адаптеры. `CourseManager` при необходимости можно оставить только как тонкий входной класс для совместимости со старым интерфейсом, но он не должен содержать бизнес-логику: все операции должны делегироваться специализированным сервисам.

Компонент	Ответственность
<code>Course</code>	Общий интерфейс курса.
<code>OnlineCourse</code> , <code>OfflineCourse</code> , <code>HybridCourse</code>	Конкретные типы курсов. Новый тип курса добавляется отдельным классом.
<code>CourseFactoryRegistry</code> , <code>CourseFactory</code> и конкретные фабрики	Выбор фабрики по типу курса и создание курса без <code>if/else</code> в бизнес-логике.
<code>CourseService</code>	Управление курсами: создание, получение, изменение. Получает фабрику через <code>CourseFactoryRegistry</code> .
<code>StudentService</code>	Работа со студентами и их данными.
<code>EnrollmentService</code>	Регистрация студента на курс, запуск оплаты и публикация событий.
<code>GradeService</code>	Расчет итоговой оценки через выбранную стратегию.
<code>GradeStrategy</code> и <code>GradeStrategySelector</code>	Выбор и выполнение алгоритма расчета итоговой оценки по типу или настройке курса.
<code>PaymentService</code>	Оплата курса через абстракцию платежного провайдера.
<code>PaymentProvider</code>	Внутренний интерфейс платежной системы.
<code>ExternalPaymentAdapter</code>	Адаптация конкретного внешнего платежного API к интерфейсу системы.
<code>PaymentProxy</code>	Контроль платежа: проверка прав, логирование и защита от повторной операции.
<code>EventPublisher</code>	Публикация доменных событий: <code>EnrollmentPaidEvent</code> , <code>GradeCalculatedEvent</code> и др.
<code>NotificationObserver</code>	Интерфейс подписчика на события уведомлений.
<code>Email/SMS/Telegram observers</code>	Отправка уведомлений разными каналами без изменения бизнес-сервисов.
<code>CourseRepository</code> , <code>StudentRepository</code> , <code>EnrollmentRepository</code> , <code>PaymentRepository</code>	Хранение и получение данных о курсах, студентах, регистрациях и платежах.

3. Диаграмма классов

UML-диаграмма отражает основные классы, интерфейсы и зависимости новой архитектуры.

Диаграмма классов в UML-нотации:

UML-диаграмма классов: новая структура системы управления курсами

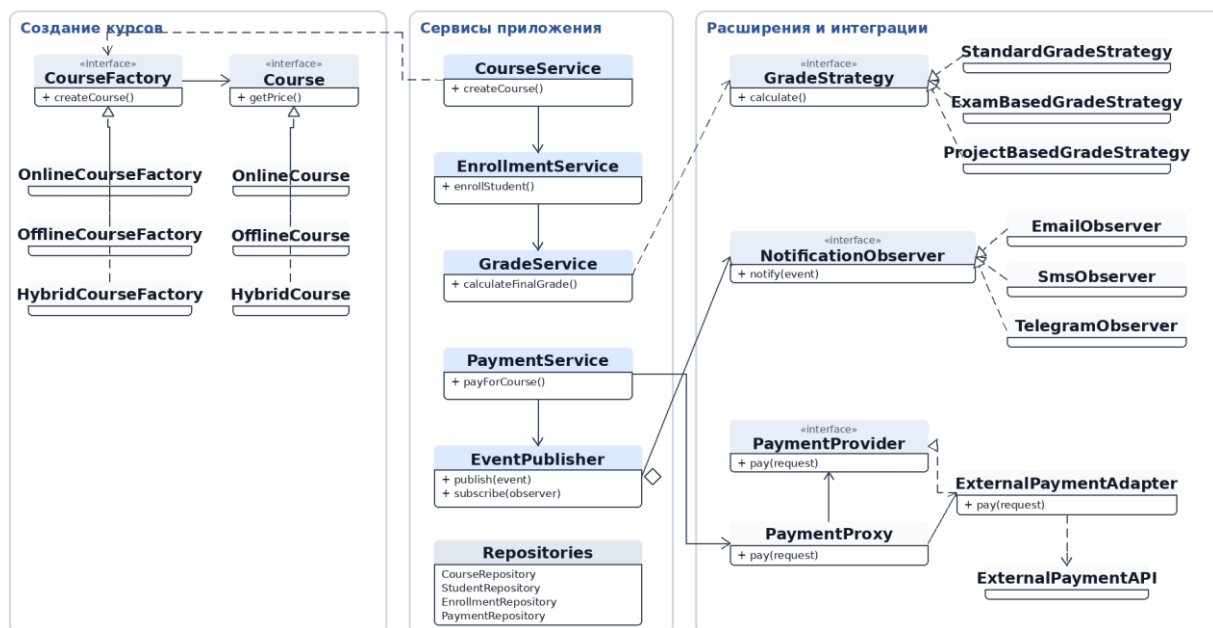


Рисунок 1 — UML-диаграмма классов новой архитектуры системы управления курсами

Легенда UML-нотации:

Обозначение на диаграмме	UML-смысл	Использование в решении
Прямоугольник с названием класса	Класс	CourseService, EnrollmentService, PaymentService и конкретные реализации.
Прямоугольник с пометкой «interface»	Интерфейс	Course, CourseFactory, GradeStrategy, NotificationObserver, PaymentProvider.
Линия с пустым треугольником	Обобщение / реализация интерфейса	OnlineCourseFactory реализует CourseFactory; OnlineCourse реализует Course.
Сплошная линия со стрелкой	Направленная ассоциация / вызов	EnrollmentService вызывает PaymentService; PaymentService использует PaymentProxy.
Пунктирная линия со стрелкой	Зависимость	ExternalPaymentAdapter обращается к ExternalPaymentAPI.
Пустой ромб у владельца связи	Агрегация	EventPublisher содержит набор подписчиков NotificationObserver.

В схеме намеренно показаны зависимости от интерфейсов: это снижает связанность и позволяет добавлять новые типы курсов, алгоритмы оценки, каналы уведомлений и платежные провайдеры без изменения основной бизнес-логики

4. Используемые паттерны проектирования

Паттерн	Где используется	Какую задачу решает
Factory Method	CourseFactoryRegistry, CourseFactory, OnlineCourseFactory, OfflineCourseFactory, HybridCourseFactory	Убирает if/else и прямое создание курсов из бизнес-логики. CourseService получает фабрику из registry по типу курса.
Strategy	GradeStrategySelector, GradeStrategy, StandardGradeStrategy, ExamBasedGradeStrategy, ProjectBasedGradeStrategy	Позволяет выбирать и менять алгоритм расчета оценки без изменения GradeService.

Observer	EventPublisher, NotificationObserver, Email/SMS/Telegram observers	Убирает прямые вызовы sendEmail(...). Сервис публикует событие, а подписчики выполняют уведомление.
Adapter	PaymentProvider, ExternalPaymentAdapter	Изолирует внешний платежный API. При смене платежного сервиса меняется адаптер, а не бизнес-логика.
Proxy	PaymentProxy перед PaymentProvider	PaymentProxy реализует PaymentProvider и оборачивает реальный провайдер, добавляя контроль доступа, логирование и защиту от повторного платежа.

5. Основная логика взаимодействия

1. Администратор создает курс через CourseService.
2. CourseService получает нужную CourseFactory из CourseFactoryRegistry по типу курса и создает объект Course без прямого вызова конструктора конкретного курса.
3. Студент регистрируется на курс через EnrollmentService; сервис проверяет студента и курс через CourseRepository и StudentRepository.
4. EnrollmentService вызывает PaymentService, а тот выполняет оплату через PaymentProxy и PaymentProvider.
5. ExternalPaymentAdapter преобразует внутренний PaymentRequest в формат внешнего платежного API.
6. После успешной оплаты EnrollmentService сохраняет регистрацию и публикует EnrollmentPaidEvent через EventPublisher.
7. Подписчики NotificationObserver реагируют на событие и отправляют уведомление нужным каналом.
8. GradeService получает GradeStrategy через GradeStrategySelector по типу или настройке курса, рассчитывает итоговую оценку и публикует GradeCalculatedEvent.

За счет такой схемы создание курсов, расчет оценок, уведомления, платежи и хранение данных не смешиваются в одном классе.

6. Соответствие проблем и решений

Проблема	Решение	Используемый паттерн / принцип	Обоснование
CourseManager перегружен обязанностями	Разделить на CourseService, EnrollmentService, GradeService, PaymentService, EventPublisher и repositories.	SRP / разделение ответственности	У каждого класса одна область ответственности.
Курсы создаются через if/else и new OnlineCourse(...)	Вынести создание в CourseFactoryRegistry и конкретные CourseFactory.	Factory Method	Сервис выбирает фабрику по типу курса, без цепочки условий.
Новый тип курса требует изменения CourseManager	Добавить новый Course и новую фабрику, затем зарегистрировать ее.	Factory Method	Расширение идет через новые классы, а не через правку старого кода.
Один жестко заданный алгоритм оценки	Ввести GradeStrategy и GradeStrategySelector.	Strategy	Алгоритм меняется без изменения GradeService.
Уведомления встроены в бизнес-логику	Публиковать события через EventPublisher и обрабатывать их подписчиками.	Observer	Новый канал добавляется отдельным observer.
Платежный API вызывается напрямую	Ввести PaymentProvider и ExternalPaymentAdapter.	Adapter	Внешний API скрыт за внутренним интерфейсом системы.
Нужны проверки, логирование и retry платежей	Добавить PaymentProxy перед PaymentProvider.	Proxy	Техническая логика вынесена из бизнес-сервиса.
Дублирование похожих действий	Общие операции вынести в сервисы, repositories и обработчики событий.	DRY / принцип	Повторяющаяся логика хранится в одном месте.
Жесткие зависимости от реализаций	Сервисы зависят от интерфейсов, а не от реализаций.	DIP / принцип	Конкретные реализации можно заменить.

7. Краткий псевдокод

Создание курса через Factory Method

```
interface CourseFactory {
    Course createCourse(CourseCreateRequest request);
}

class CourseFactoryRegistry {
    private Map<CourseType, CourseFactory> factories;

    CourseFactory getFactory(CourseType type) {
        return factories.get(type);
    }
}

class OnlineCourseFactory implements CourseFactory {
    Course createCourse(request) {
        return new OnlineCourse(request.title, request.price, request.link);
    }
}

class CourseService {
    Course createCourse(request) {
        CourseFactory factory = registry.getFactory(request.type);
        Course course = factory.createCourse(request);
        repository.save(course);
        return course;
    }
}
```

Расчет оценки через Strategy

```
interface GradeStrategy {
    Grade calculate(Student student, Course course);
}

class ExamBasedGradeStrategy implements GradeStrategy {
    Grade calculate(student, course) {
        return calculateByExamResult(student, course);
    }
}

class GradeStrategySelector {
    GradeStrategy select(Course course) {
        return strategies.get(course.gradePolicy());
    }
}

class GradeService {
    Grade calculateFinalGrade(student, course) {
        GradeStrategy strategy = selector.select(course);
        return strategy.calculate(student, course);
    }
}
```

Оплата через Adapter и Proxy

```
interface PaymentProvider {
    PaymentResult pay(PaymentRequest request);
}

class ExternalPaymentAdapter implements PaymentProvider {
    PaymentResult pay(request) {
        return map(externalApi.makePayment(request));
    }
}

class PaymentProxy implements PaymentProvider {
    PaymentResult pay(request) {
        checkAccess(request);
    }
}
```

```

        checkDuplicatePayment(request);
        logPaymentAttempt(request);
        return provider.pay(request);
    }
}

```

Уведомления через Observer

```

interface NotificationObserver { void notify(DomainEvent event); }

class EmailNotificationObserver implements NotificationObserver {
    void notify(event) { emailClient.send(event.message); }
}

class EventPublisher {
    void publish(event) { observers.forEach(o -> o.notify(event)); }
}

```

8. Соблюдение DRY, KISS и разделения ответственности

Принцип	Как соблюден
DRY	Повторяющиеся операции вынесены в отдельные сервисы, repositories и общие интерфейсы.
KISS	Архитектура состоит из понятных компонентов: сервисы, фабрики, стратегии, адаптеры, observers. Лишние абстракции не добавляются.
Разделение ответственности	Создание курсов, расчет оценок, уведомления, оплата и хранение данных разделены по разным классам.
SRP	Каждый сервис имеет одну основную причину для изменения.
OCP	Новые типы курсов, алгоритмы оценок и каналы уведомлений добавляются через новые реализации, а не через изменение существующего кода.
DIP	Сервисы зависят от интерфейсов CourseFactory, GradeStrategy, PaymentProvider и NotificationObserver.

Проверка расширяемости:

Изменение	Что нужно сделать
Добавить вебинарный курс	Создать WebinarCourse и WebinarCourseFactory; зарегистрировать фабрику в CourseFactoryRegistry.
Добавить расчет оценки по тестам	Создать TestBasedGradeStrategy и подключить ее через GradeStrategySelector.
Добавить Telegram-уведомления	Создать TelegramNotificationObserver и подписать его на события.
Сменить платежный сервис	Создать новый PaymentAdapter под новый внешний API.

9. Результат работы

Предложенная архитектура заменяет перегруженный CourseManager набором специализированных компонентов. Factory Method и CourseFactoryRegistry отвечают за создание курсов без if/else, Strategy и GradeStrategySelector — за выбор алгоритма оценки, Observer — за уведомления, Adapter — за изоляцию платежного API, Proxy — за контроль платежей. Каждая проблема исходной системы имеет отдельное архитектурное решение. В результате система становится расширяемой, зависимости изолируются, дублирование уменьшается, а основная бизнес-логика остается простой и понятной.